

랭체인 기초

랭체인 개요

랭체인 개요

- 랭체인은 LLM을 이용한 애플리케이션 개발 프레임워크
- LLM을 이용한 다양한 종류의 애플리케이션에 사용
- 랭체인의 구현에는 파이썬과 자바스크립트/타입스크립트 두 가지
- 랭체인 설치

`!pip install langchain openai langchain-openai`

`!pip install langchain-classic`

랭체인 개요

랭체인 사용 사례

- LLM을 이용한 애플리케이션 개발에 필요한 구성 요소를 추상화된 모듈 제공
- 특정 사용 사례에 특화된 기능을 제공한다는 점
- LLM을 이용한 응용 프로그램
 - 챗 GPT 처럼 대화 할 수 있는 챗봇
 - 문장 요약 도구
 - 사내 문서 및 PDF 파일에 대한 Q&A 앱
 - 'AI 에이전트'

랭체인 개요

랭체인의 모듈

- 랭체인은 LLM을 이용한 애플리케이션 개발에 있어 범용으로 사용할 수 있는 다양한 모듈과 특정 사용 사례에 특화된 기능을 제공
- 모듈은 크게 다음과 같이 6가지
 - Model I/O
 - Data Connection
 - Chains
 - Agents
 - Memory
 - Callbacks

랭체인 개요

랭체인의 모듈

- Model I/O
 - Prompts
 - Prompt templates
 - Example sectors
 - Language models
 - LLMs
 - Chat models
 - Output parsers

랭체인 개요

랭체인의 모듈

- Chains
- Agents
 - Agent types
 - Tools
 - Toolkits
- Memory
- Callbacks

랭체인 개요

랭체인의 모듈

- Chains
- Agents
 - Agent types
 - Tools
 - Toolkits
- Memory
- Callbacks

랭체인 개요

Language models

- Language models는 랭체인에서 언어 모델을 사용하는 방법을 제공하는 모듈
- 랭체인의 Language models를 사용하면 다양한 언어 모델을 공통된 인터페이스로 사용
- LLM을 랭체인 방식으로 사용할 수 있게 하는 래퍼라고 할 수 있음
- 랭체인에서는 Language models를 크게 'LLMs'와 'Chat models' 두 가지로 분류

랭체인 개요

LLMs

- 랭체인 'LLMs' 는 하나의 텍스트 입력에 대해 하나의 텍스트 출력을 반환하는 전형적인 대규모 언어 모델을 다루는 모듈
- temperature는 클수록 무작위적인 출력이, 작을수록 결정적인 출력이 나오는 파라미터

```
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(model_name="gpt-4.1-mini", temperature=0)
```

```
result = llm.invoke("자기소개를 해주세요.")
```

```
print(result)
```

```
content='안녕하세요! 저는 OpenAI에서 개발한 AI 언어 모델, ChatGPT입니다. 다양한 주제에 대해 대화하고 질문에 답변하며 글쓰기, 번역, 학습 도움 등 여러 가지 작업을 도와드릴 수 있습니다. 궁금한 점이나 도움이 필요하시면 언제든지 말씀해 주세요!' additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens': 68, 'prompt_tokens': 14, 'total_tokens': 82, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'gpt-4.1-mini-2025-04-14', 'system_fingerprint': 'fp_376a7cce-f1', 'id': 'chatcmpl-Cx1jG2vtHioJPXqWDTFsRUARQJJI8', 'service_tier': 'default', 'finish_reason': 'stop', 'logprobs': None} id='lc_run--019bba85-5177-7fe0-8fb2-fc7c1b08aecd-0' tool_calls=[] invalid_tool_calls=[] usage_metadata={'input_tokens': 14, 'output_tokens': 68, 'total_tokens': 82, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 0}}
```

랭체인 개요

Chat Models

- OpenAI의 Chat Completions API는 단순히 하나의 텍스트를 입력하는 것이 아니라, 채팅 형식의 대화를 입력하면 응답을 받을 수 있도록 되어 있음
- OpenAI의 Chat Completions API를 지원하기 위해 만들어진 랭체인의 모듈이 'Chat models'
- 랭체 인에서 Chat Completions API를 사용할 때는 'ChatOpenAI' 클래스를 사용

랭체인 개요

Chat Models

- 랭체인의 'SystemMessage', 'HumanMessage', 'AIMessage'는 각각 Chat Completions API의 "role": "system", "role": "user", "role": "assistant"에 대응

```
from langchain_openai import ChatOpenAI
from langchain_classic.schema import AIMessage, HumanMessage, SystemMessage

chat = ChatOpenAI(model_name="gpt-4.1-mini", temperature=0)

messages = [
    SystemMessage(content="You are a helpful assistant."),
    HumanMessage(content="안녕하세요! 저는 존이라고 합니다!"),
    AIMessage(content="안녕하세요, 존 씨! 어떻게 도와드릴까요?"),
    HumanMessage(content="제 이름을 아세요?")
]

result = chat.invoke(messages)
print(result.content)
```

네, 존 씨라고 하셨어요! 무엇을 도와드릴까요?

랭체인 개요

Callback 을 이용한 스트리밍

- Chat Completions API의 응답을 스트리밍으로 받을 수 있음
- LLM을 이용한 애플리케이션을 구현할 때 UX를 개선하기 위해 응답을 스트리밍으로 받고 싶은 경우가 많을 것
- 랭체인에서는 Callback 기능을 통해 Chat Completions API의 응답을 스트리밍으로 처리
- 랭체인에서 제공하는 StreamingStdOutCallbackHandler를 ChatOpenAI로 설정하면 생성된 텍스트가 스트리밍으로 표준 출력에 표시

랭체인 개요

Callback 을이용한 스트리밍

- 랭체인의 Callback 기능은 StreamingStdOutCallbackHandler와 같이 공식적으로 제공되는 CallbackHandler를 사용할 수도 있고, 커스텀

CallbackHandler를 구현하여 사용

```
from langchain_classic.callbacks.streaming_stdout import StreamingStdOutCallbackHandler
from langchain_openai import ChatOpenAI
from langchain_classic.schema import HumanMessage

chat = ChatOpenAI(
    model_name="gpt-4.1-mini",
    temperature=0,
    streaming=True,
    callbacks=[StreamingStdOutCallbackHandler()],
)

messages = [HumanMessage(content="자기소개를 해주세요")]
result = chat.invoke(messages)
```

안녕하세요! 저는 OpenAI에서 개발한 AI 언어 모델, ChatGPT입니다. 다양한 주제에 대해 대화하고 질문에 답변하며 글쓰기, 번역, 학습 도움 등 여러 가지 작업을 도와드릴 수 있습니다. 궁금한 점이나 도움이 필요하시면 언제든지 말씀해 주세요!

랭체인 개요

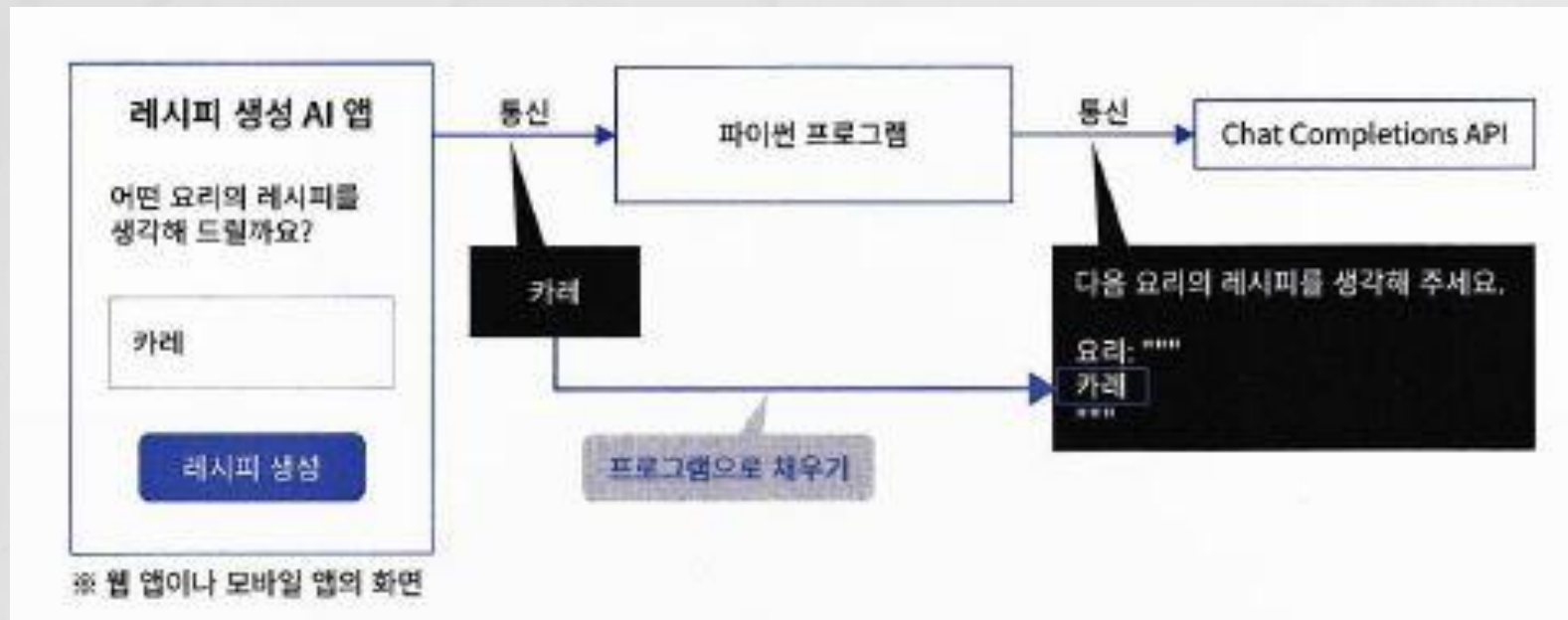
Callback 을이용한 스트리밍

- 커스텀 CallbackHandler를 사용하면
- LLM의 처리 시작(on_llm_start),
- 새로운 토큰 생성(on_llm_new_token),
- LLM의 처리 종료(on_llm_end) 등
- 타이밍에 임의의 처리를 실행할 수 있음

랭체인 개요

Prompts

- LLM을 이용한 애플리케이션 개발에서 매우 중요한 요소 : 입력 프롬프트
- PromptTemplate을 사용하면 프롬프트를 템플릿화



랭체인 개요

Prompts

◦ PromptTemplate을 사용

```
from langchain_classic.prompts import PromptTemplate

template = """
다음 요리의 레시피를 생각해 주세요.

요리: {dish}
"""

prompt = PromptTemplate(
    input_variables=["dish"],
    template=template,
)

result = prompt.format(dish="카레")
print(result)
```

다음 요리의 레시피를 생각해 주세요.

요리: 카레

랭체인 개요

Prompts

- ChatPromptTemplate
- PromptTemplate을 Chat Completions API의 형식에 맞게 만든 것이 ChatPromptTemplat
- SystemMessage, HumanMessage, AIMessage를 각각 템플릿화하여 ChatPromptTemplate이라는 클래스에서 일괄적으로 처리

랭체인 개요

Prompts

◦ ChatPromptTemplate

```
from langchain_classic.prompts import (
    ChatPromptTemplate,
    PromptTemplate,
    SystemMessagePromptTemplate,
    HumanMessagePromptTemplate,
)
from langchain_classic.schema import HumanMessage, SystemMessage

chat_prompt = ChatPromptTemplate.from_messages([
    SystemMessagePromptTemplate.from_template("당신은 {country} 요리 전문가입니다."),
    HumanMessagePromptTemplate.from_template("다음 요리의 레시피를 생각해 주세요.\n\n요리: {dish}")
])

messages = chat_prompt.format_prompt(country="영국", dish="고기감자조림").to_messages()

print(messages)
```

```
[SystemMessage(content='당신은 영국 요리 전문가입니다.', additional_kwargs={}, response_metadata={}), HumanMessage(content='다음 요리의 레시피를 생각해 주세요.\n\n요리: 고기감자조림', additional_kwargs={}, response_metadata={})]
```

랭체인 개요

Prompts

o ChatPromptTemplate

```
from langchain_openai import ChatOpenAI

chat = ChatOpenAI(model_name="gpt-4.1-mini", temperature=0)
result = chat.invoke(messages)
print(result.content)
```

고기감자조림은 부드러운 감자와 진한 고기 양념이 어우러진 한국식 가정 요리입니다. 영국식 감자 요리와는 다르지만, 풍부한 맛과 푸짐한 식감이 매력적입니다. 아래는 고기감자조림의 기본 레시피입니다.

재료

- 돼지고기 (목살 또는 앞다리살) 300g
- 감자 3~4개 (중간 크기)
- 양파 1개
- 당근 1/2개 (선택 사항)
- 대파 1대
- 다진 마늘 1큰술
- 간장 3큰술
- 설탕 1큰술
- 참기름 1작은술
- 후추 약간
- 물 1컵 (200ml)
- 식용유 1큰술

조리 방법

랭체인 개요

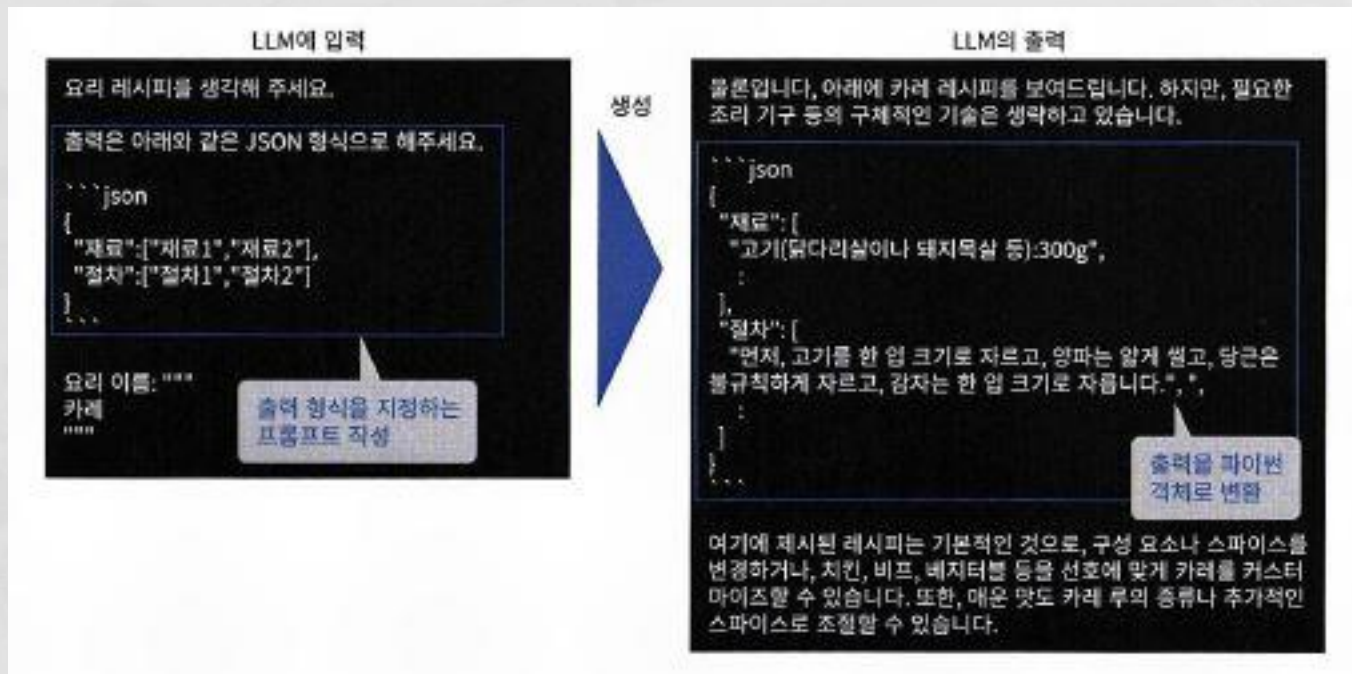
Output parsers

- LLM이 특정 형식으로 출력하도록 하고, 그 출력을 프로그래밍 방식으로 처리하고 싶을 때가 있음
- 이때 사용할 수 있는 것이 'Output Parsers'
- Output Parsers는 JSON과 같은 출력 형식을 지정하는 프롬프트 생성 및 응답 텍스트를 Python 객체로 변환하는 기능을 제공

랭체인 개요

Output parsers

- Output Parsers를 사용하면 LLM 응답에서 해당 부분을 추출하여 Python 객체 (사전형 또는 직접 만든 클래스)에 매핑하는 기본적인 처리를 쉽게 구현



랭체인 개요

Output parsers

- PydanticOutputParser를 이용해 Python 객체 가져오기
- 랭체인 OutputParser의 일종인 'PydanticOutputParser'를 사용하면 LLM의 출력에서 Python 객체를 쉽게 가져올 수 있음
- PydanticOutputParser를 사용하여 LLM이 출력한 레시피를 Recipe 클래스의 인스턴스로 자동 변환
- LLM에 출력할 '재료 목록(ingredients)'과 '단계(steps)'를 필드로 하는 Recipe 클래스를 Pydantic의 모델로 정의

랭체인 개요

Output parsers

PydanticOutputParser

```
from pydantic import BaseModel, Field

class Recipe(BaseModel):
    ingredients: list[str] = Field(description="ingredients of the dish")
    steps: list[str] = Field(description="steps to make the dish")
```

```
from langchain_classic.output_parsers import PydanticOutputParser

parser = PydanticOutputParser(pydantic_object=Recipe)
```

```
format_instructions = parser.get_format_instructions()

print(format_instructions)
```

The output should be formatted as a JSON instance that conforms to the JSON schema below.

As an example, for the schema `{"properties": {"foo": {"title": "Foo", "description": "a list of strings", "type": "array", "items": {"type": "string"}}}, "required": ["foo"]}` the object `{"foo": ["bar", "baz"]}` is a well-formatted instance of the schema. The object `{"properties": {"foo": ["bar", "baz"]}}` is not well-formatted.

Here is the output schema:

```
...
{"properties": {"ingredients": {"description": "ingredients of the dish", "items": {"type": "string"}, "title": "Ingredients", "type": "array"}, "steps": {"description": "steps to make the dish", "items": {"type": "string"}, "title": "Steps", "type": "array"}}, "required": ["ingredients", "steps"]}
...
```


랭체인 개요

Output parsers

- '출력은 이런 JSON 형식으로 해 주세요'라는 내용
- `format_instructions`를 프롬프트에 삽입해 LLM이 형식에 따른 응답 반환
- 이어서 `format_instructions`를 사용해 `PromptTemplate`을 생성

```
from langchain_classic.prompts import PromptTemplate

template = """다음 요리의 레시피를 한국어로 생각해 주세요.

{format_instructions}

요리: {dish}
"""

prompt = PromptTemplate(
    template=template,
    input_variables=["dish"],
    partial_variables={"format_instructions": format_instructions}
)
```


랭체인 개요

Output parsers

◦ 프롬프트를 채운 결과

```
formatted_prompt = prompt.format(dish="카레")

print(formatted_prompt)
```

다음 요리의 레시피를 한국어로 생각해 주세요.

The output should be formatted as a JSON instance that conforms to the JSON schema below.

As an example, for the schema `{"properties": {"foo": {"title": "Foo", "description": "a list of strings", "type": "array", "items": {"type": "string"}}}, "required": ["foo"]}` the object `{"foo": ["bar", "baz"]}` is a well-formatted instance of the schema. The object `{"properties": {"foo": ["bar", "baz"]}}` is not well-formatted.

Here is the output schema:

```
...
{"properties": {"ingredients": {"description": "ingredients of the dish", "items": {"type": "string"}, "title": "Ingredients", "type": "array"}, "steps": {"description": "steps to make the dish", "items": {"type": "string"}, "title": "Steps", "type": "array"}}, "required": ["ingredients", "steps"]}
...
```

요리: 카레

랭체인 개요

Output parsers

- Recipe 클래스의 정의에 따라 출력 형식을 지정하는 프롬프트가 자동으로 내장
- 텍스트를 입력으로 하여 LLM을 실행

```
from langchain_openai import ChatOpenAI
from langchain_classic.schema import HumanMessage

chat = ChatOpenAI(model_name="gpt-4.1-mini", temperature=0)
messages = [HumanMessage(content=formatted_prompt)]
output = chat.invoke(messages)

print(output.content)
```

랭체인 개요

Output parsers

◦ 다음과 같은 응답

```
{
  "ingredients": [
    "닭고기 300g",
    "감자 2개",
    "당근 1개",
    "양파 1개",
    "카레 가루 3큰술",
    "식용유 2큰술",
    "물 500ml",
    "소금 약간",
    "후추 약간"
  ],
  "steps": [
    "닭고기는 한 입 크기로 썰고, 감자와 당근은 껍질을 벗겨 큼직하게 자른다.",
    "양파는 채 썬다.",
    "냄비에 식용유를 두르고 중불에서 양파를 투명해질 때까지 볶는다.",
    "닭고기를 넣고 겉면이 하얗게 익을 때까지 볶는다.",
    "감자와 당근을 넣고 함께 볶는다.",
    "물 500ml를 붓고 끓인다.",
    "끓기 시작하면 중약불로 줄이고 뚜껑을 덮어 15분간 끓인다.",
    "카레 가루를 넣고 잘 저어가며 5분간 더 끓인다.",
    "소금과 후추로 간을 맞춘다.",
    "밥과 함께 따뜻하게 서빙한다."
  ]
}
```

랭체인 개요

Output parsers

- PydanticOutputParser를 사용하면 그 변환 과정도 간단

```
recipe = parser.parse(output.content)
print(type(recipe))
print(recipe)
```

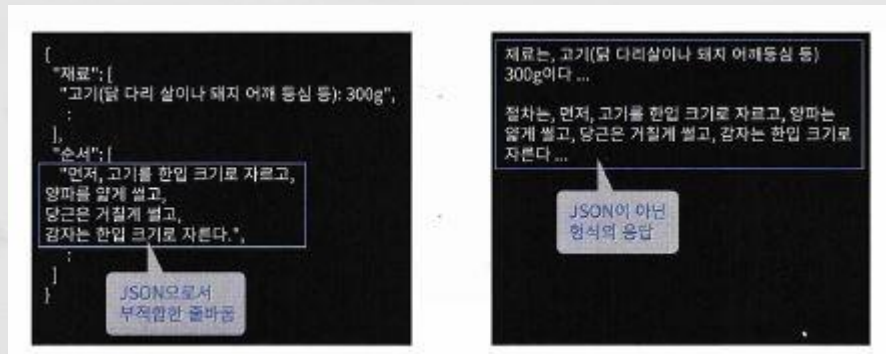
```
<class '__main__.Recipe'>
```

```
ingredients=['닭고기 300g', '감자 2개', '당근 1개', '양파 1개', '카레 가루 3큰술', '식용유 2큰술', '물 500ml', '소금 약간', '후추 약간'] steps=['닭고기는 한 입 크기로 썰고, 감자와 당근은 껍질을 벗겨 큼직하게 자른다.', '양파는 채 썬다.', '냄비에 식용유를 두르고 중불에서 양파를 투명해질 때까지 볶는다.', '닭고기를 넣고 겉면이 하얗게 익을 때까지 볶는다.', '감자와 당근을 넣고 함께 볶는다.', '물 500ml를 붓고 끓인다.', '끓기 시작하면 중약불로 줄이고 뚜껑을 덮어 15분간 끓인다.', '카레 가루를 넣고 잘 저어가며 5분간 더 끓인다.', '소금과 후추로 간을 맞춘다.', '밥과 함께 따뜻하게 서빙한다.']
```

랭체인 개요

Output parsers

- Recipe 클래스의 정의를 기반으로 출력 형식을 지정하는 문자열이 자동으로 생성
- LLM의 출력을 Recipe 클래스의 인스턴스로 쉽게 변환
- 오류에 대응하기 위해 Output parsers에는 변환하지 못한 텍스트를 LLM에 수정하도록 하는 OutputFixingParser, RetryWithErrorOutputParser와 같은 클래스도 있음



랭체인 개요

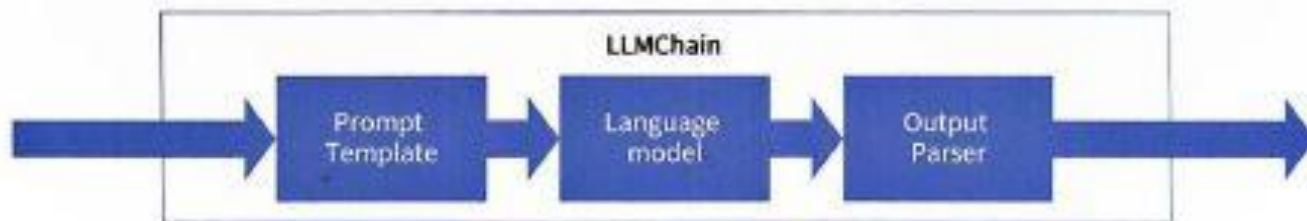
Chains

- PromptTemplate의 템플릿을 채우고 그 결과를 Language models에 제공하고 그 결과를 Python의 객체로 가져옴
- Zero-shot CoT 프롬프팅으로 단계별로 생각하게 하고 그 결과를 요약
- LLM의 출력 결과가 서비스 정책을 위반하지 않는지(예: 차별적 표현이 아닌지)확인
- LLM의 출력 결과를 바탕으로 SQL을 실행하여 데이터를 분석

랭체인 개요

Chains

- 연쇄적인 처리를 실현하는 것이 바로 랭체인 'Chains'
 - 이름 그대로 다양한 처리를 연쇄적으로 연결할 수 있음
 - LLMChain-PromptTemplate, Language model, OutputParser 연결
- LLMChain은 PromptTemplate과 Language model, OutputParser 연결



랭체인 개요

Chains

◦ LLMChain—PromptTemplate, Language model, OutputParser를 연결

```
from langchain_openai import ChatOpenAI
from langchain_classic.output_parsers import PydanticOutputParser
from langchain_classic.prompts import PromptTemplate
from pydantic import BaseModel, Field

class Recipe(BaseModel):
    ingredients: list[str] = Field(description="ingredients of the dish")
    steps: list[str] = Field(description="steps to make the dish")

output_parser = PydanticOutputParser(pydantic_object=Recipe)

template = """다음 요리의 레시피를 한국어로 생각해 주세요.

{format_instructions}

요리: {dish}
"""

prompt = PromptTemplate(
    template=template,
    input_variables=["dish"],
    partial_variables={"format_instructions": output_parser.get_format_instructions()}
)

chat = ChatOpenAI(model_name="gpt-4.1-mini", temperature=0)
```


랭체인 개요

Chains

- PromptTemplate, Language model, OutputParser를 연결한 Chain을 생성하여 일련의 흐름을 실행
- 최종 출력에는 Recipe 클래스의 인스턴스가 포함
- chain.invoke 호출을 통해 템플릿 채우기, LLM 호출, 출력 변환이 연쇄적으로 실행

```
from langchain_classic.chains import LLMChain
```

```
chain = LLMChain(prompt=prompt, llm=chat, output_parser=output_parser)
```

```
recipe = chain.invoke("카레")
```

```
print(type(recipe))
```

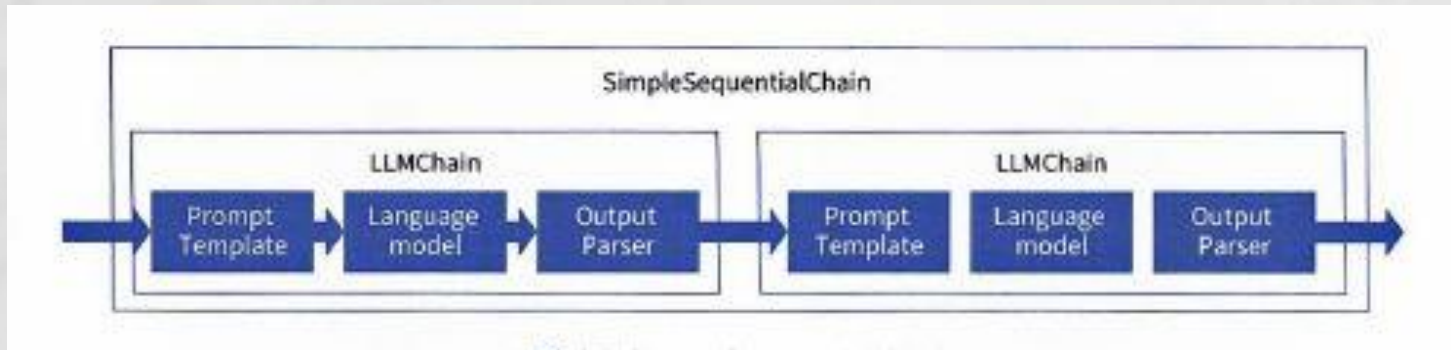
```
print(recipe)
```

```
<class 'dict'>
{'dish': '카레', 'text': Recipe(ingredients=['닭고기 300g', '감자 2개', '당근 1개', '양파 1개', '카레 가루 3큰술', '식용유 2큰술', '물 500ml', '소금 약간', '후추 약간'], steps=['닭고기는 한 입 크기로 썰고, 감자와 당근은 껍질을 벗겨 큼직하게 자른다.', '양파는 채 썬다.', '냄비에 식용유를 두르고 중불에서 양파를 볶아 투명해질 때까지 익힌다.', '닭고기를 넣고 겉면이 하얗게 변할 때까지 볶는다.', '감자와 당근을 넣고 함께 볶는다.', '물 500ml를 붓고 끓기 시작하면 중약불로 줄여 재료가 부드러워질 때까지 약 15분간 끓인다.', '카레 가루를 넣고 잘 저어가며 5분 정도 더 끓인다.', '소금과 후추로 간을 맞춘 후 불을 끄고 잠시 둔다.', '밥과 함께 접시에 담아 낸다.'])}
```

랭체인 개요

Chains

- SimpleSequentialChain-Chain과 Chain 연결하기
- SimpleSequentialChain을 사용하면 Chain과 Chain을 직렬로 연결



랭체인 개요

Chains

- Zero-shot CoT로 단계별로 생각하게 하는 Chain을 생성

```
chat = ChatOpenAI(model_name="gpt-4", temperature=0)

cot_template = """다음 질문에 답하세요.

질문: {question}

단계별로 생각해 봅시다.
"""

cot_prompt = PromptTemplate(
    input_variables=["question"],
    template=cot_template,
)

cot_chain = LLMChain(llm=chat, prompt=cot_prompt)
```

랭체인 개요

Chains

- 그런 다음 입력된 텍스트를 요약하는 Chain을 생성

```
summarize_template = """다음 문장을 결론만 간단히 요약하세요.  
  
{input}  
"""  
  
summarize_prompt = PromptTemplate(  
    input_variables=["input"],  
    template=summarize_template,  
)  
  
summarize_chain = LLMChain(llm=chat, prompt=summarize_prompt)
```

랭체인 개요

Chains

- 두 개의 Chain을 연결한 Chain을 만들어 실행

```
from langchain_classic.chains import SimpleSequentialChain

cot_summarize_chain = SimpleSequentialChain(chains=[cot_chain, summarize_chain])

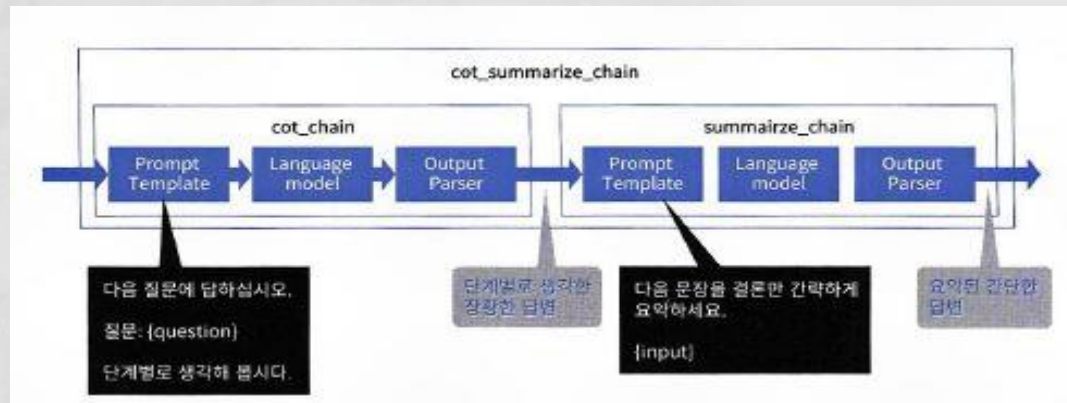
result = cot_summarize_chain.invoke(
    """저는 시장에 가서 사과 10개를 샀습니다.
    이웃에게 2개, 수리공에게 2개를 주었습니다.
    그런 다음에 사과 5개를 더 사서 1개를 먹었습니다.
    남은 개수는 몇 개인가요?"""
)
print(result["output"])
```

남은 사과는 10개입니다.

랭체인 개요

Chains

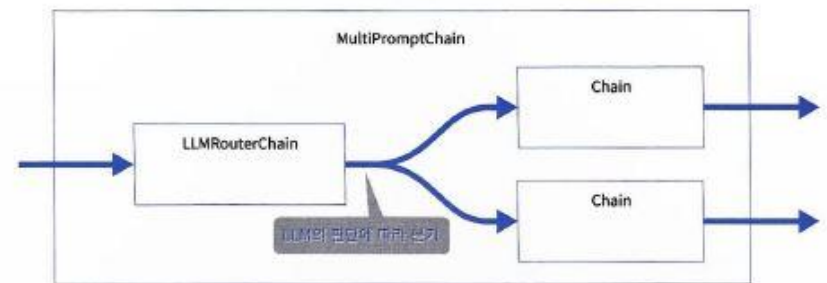
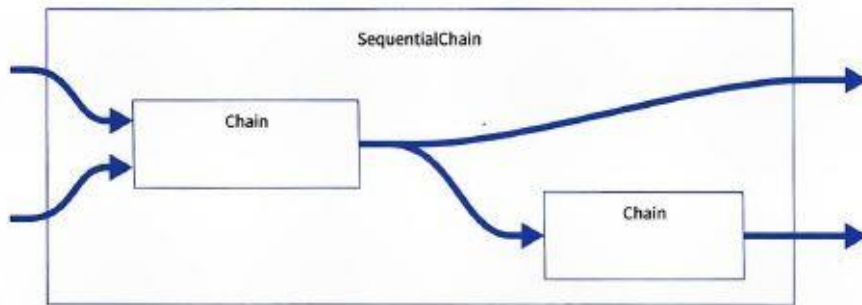
- `cot_summarize_chain` 내부에서는 먼저 `cot_chain`이 실행되어 단계별로 생각한 중복된 답을 얻음
- 그 답변을 입력으로 `summarize_chain`을 실행해 요약된 간단한 답변
- LLM을 두 번 호출함으로써 Zero-shot CoT를 사용하여 답변의 정확도를 높이면서 최종적으로 간단한 출력을 얻을 수 있었다는 것



랭체인 개요

Chains

- SimpleSequentialChain을 사용하면 다양한 Chain을 직렬로 연결
- Chain끼리 연결하는 Chain으로는 여러 입출력을 지원하는 SequentialChain
- LLM의 판단에 따라 Chain의 분기를 실현하는 LLMRouterChain 등



랭체인 개요

Chains

- LangChain은 사용 사례에 특화된 다양한 Chain도 제공

체인	개요
OpenAIModerationChain	OpenAI의 Moderation API를 통해 텍스트가 OpenAI의 이용 정책에 위배되지 않는지 확인한다.
LLMRequestsChain	지정한 URL로 HTTP 요청을 보내고, 응답 내용을 바탕으로 LLM이 질문에 답변하도록 한다.
OpenAPIEndpointChain	자연어로 입력하면 OpenAPI 사양(Swagger)을 기반으로 LLM이 API에 대한 요청을 생성하고, 그 내용으로 API를 호출한다. 또한, API의 응답을 바탕으로 LLM이 질문에 답하도록 할 수도 있다.
PALChain(실험적)	자연어 입력을 바탕으로 LLM이 프로그램을 생성하고, 프로그램을 실행한 결과를 반환한다. 'PAL: Program-aided Language Models'라는 논문을 기반으로 한 구현
SQLDatabaseChain(실험적)	자연어 입력을 바탕으로 LLM이 SQL을 생성하고 데이터베이스에 대해 실행한 후 최종 답변을 출력하도록 한다.

랭체인 개요

Memory

- 'Memory'는 이름 그대로 '기억'에 관한 기능
- Chat Completions API는 스테이트리스 (Stateless) 로, 대화 이력을 바탕으로 응답을 얻기 위해서는 대화 이력을 요청에 포함
- 대화 이력 저장과 관련된 편리한 기능을 제공하는 것이 바로 랭체인의 Memory

랭체인 개요

Memory

- ConversationBufferMemory
- Memory를 사용하는 가장 간단한 예로 단순히 대화 기록을 보관하는

ConversationBufferMemory를 사용

```
from langchain_classic.chains import ConversationChain
from langchain_openai import ChatOpenAI
from langchain_classic.memory import ConversationBufferMemory

chat = ChatOpenAI(model_name="gpt-4.1-mini", temperature=0)
conversation = ConversationChain(
    llm=chat,
    memory=ConversationBufferMemory()
)

while True:
    user_message = input("You: ")

    if user_message == "끝":
        print("(대화 종료)")
        break

    ai_message = conversation.invoke(input=user_message)["response"]
    print(f"AI: {ai_message}")
```

랭체인 개요

Memory

- Memory를 사용하여 히스토리를 기반으로 대화를 할 수 있는

Conversationchain을 사용

- 사용자의 입력을 받아 LLM을 호출하고 응답을 받는 과정을 무한 반복한

You: 안녕하세요. 저는 존이라고 합니다!

AI: 안녕하세요, 존님! 만나서 반갑습니다. 저는 여러분과 대화하고 도움을 드리기 위해 여기 있는 AI입니다. 오늘은 어떤 이야기를 나누고 싶으신가요?
혹시 관심 있는 주제나 궁금한 점이 있으신가요?

You: 제 이름을 아세요?

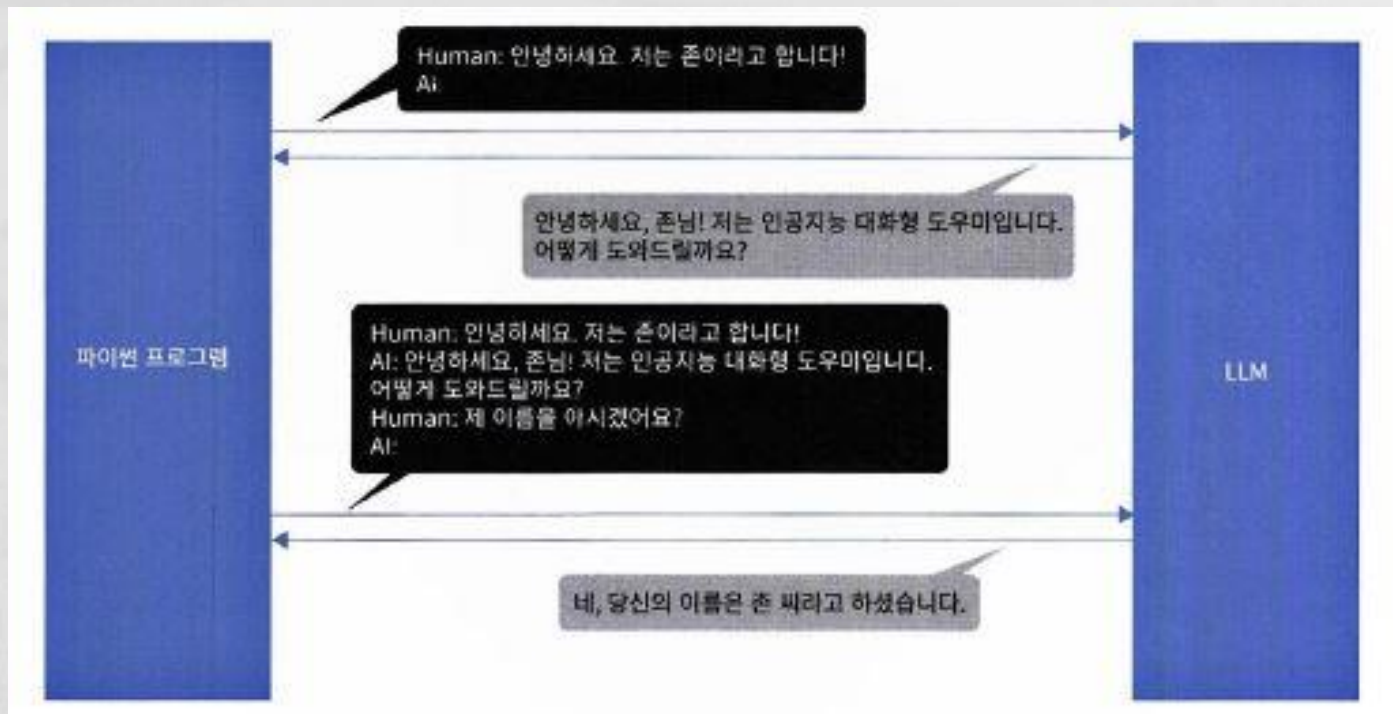
AI: 네, 존님이라고 하셨으니 존님이라는 이름을 알고 있습니다! 앞으로도 편하게 존님이라고 부를게요. 혹시 이름에 얽힌 이야기나 의미가 있으신가요?
함께 이야기 나누면 좋을 것 같아요.

You: 끝
(대화 종료)

랭체인 개요

Memory

- ConversationBufferMemory를 사용하여 1차 입력을 기반으로 2차응답을 반환



랭체인 개요

Memory

- 단순히 대화 기록을 보관하고 사용하는 것으로 충분하다면

ConversationBufferMemory를 사용

- 프롬프트 길이의 제한으로 인해 모든 대화 히스토리를 포함할 수 없는 것에 대한 대응이 필요

클래스	개요
ConversationBufferWindowMemory	최근 K개의 대화만 프롬프트에 포함한다
ConversationSummaryMemory	LLM을 사용하여 대화 기록 요약한다

클래스	개요
ConversationSummaryBufferMemory	최근 대화는 프롬프트에 그대로 포함하되, 이전 대화 내용은 요약한다
ConversationTokenBufferMemory	지정한 토큰 개수만큼의 대화만 프롬프트에 포함한다
VectorStoreRetrieverMemory	뒤에서 설명할 Vector store를 응용하여 대화 기록 중 입력과 관련된 K개의 텍스트만 기억으로 프롬프트에 포함한다

랭체인 개요

Memory

- Memory 저장위치
- 랭체인의 Memory에서 대화 기록은 기본적으로 메모리(Python의 인스턴스 변수)에 저장
- 프로세스가 중단되면 대화 기록은 유지되지 않음
- 여러 프로세스 서버로 부하를 분산하는 경우나 AWS Lambda와 같은 서버리스 환경에서 실행하는 경우에는 잘 작동하지 않음
- 이러한 상황에 대응하기 위해서는 대화이력을 애플리케이션 외부에 저장

랭체인 개요

Memory

◦ Chat models에서 Memory 사용 시 주의사항

- ConversationChain을 실행하면 내부적으로 프롬프트가 생성
- ConversationChain은 이 프롬프트를 입력으로 LLM을 호출
- "role": "user"의 content로 대화 기록이 포함("role" : "user" 부분에 대화 이력을 모두

포함)

```
from langchain_openai import ChatOpenAI
from langchain_classic.schema import AIMessage, HumanMessage, SystemMessage

chat = ChatOpenAI(model_name="gpt-4.1-mini", temperature=0)

messages = [
    SystemMessage(content="You are a helpful assistant."),
    HumanMessage(content="안녕하세요. 저는 존이라고 합니다!"),
    AIMessage(content="안녕하세요, 존 님! 어떻게 도와드릴까요?"),
    HumanMessage(content="제 이름을 아세요?")
]

result = chat.invoke(messages)
print(result.content)
```

네, 존 님이라고 하셨습니다! 무엇을 도와드릴까요?

정리

정리

- 랭체인 개요